Experiment 14 — Bioinformatics: the integer half of the buffer

Mathilda — execution-speed experiments

Contents

Experiment 14 — Bioinformatics: the integer half of the buffer

Date: 2026-07-31 · Code: `src/funcprog.c`, `src/ndkernels.c`, `src/ndarray.c`, `src/ndstruct.c`, `src/pack.c` · Result: Needleman-Wunsch 11.11 s → 186 ms, now 9.9× faster than Mathematica; k-mer counting 492 → 25.0 ms, 2.0× faster than Mathematica

Common method in `README.md`.

Hypothesis

Experiment 5 gave Mathilda an `int64` array dtype and experiments 10 and 11 put the float64 elementwise, scan, structural and convolution paths on the buffer. Nothing since has run a workload that is integer end to end.

Sequence analysis is exactly that. Bases are symbols, scores are small integers, gap penalties are integers, k-mers are integers, and an alignment matrix is integer dynamic programming. If the `int64` arms of those paths are missing, an integer workload finds it and a floating-point one never will.

The kernels

id	kernel	what it stresses
`nwalign`	Needleman-Wunsch, 2000 × 2000	integer DP: elementwise `Max`, a max-plus scan, leading-axis slic
`kmer`	12-mer encode + distinct count, 5 × 10^5 bases	sliding window, integer `Dot`, `Union`
`gcwin`	rolling GC content, 10^7 bases, window 1000	`Accumulate` + two `Drops`

The alignment is written the way a vectorising implementer writes it. The within-row gap recurrence

`h[j] = max(cand[j], h[j−1] − g)`

is a max-plus prefix scan; substituting `H[j] = h[j] + g·j` turns it into a plain running maximum, so all three systems run that — `FoldList[Max, …]` in the two CAS, `np.maximum.accumulate` in NumPy. The columns compare execution, not cleverness.

What the first run found

	Mathilda	Mathematica	NumPy
Needleman-Wunsch, 2000 × 2000	11.11 s	1.84 s	51.4 ms
k-mer, 5 × 10^5 bases, k = 12	492 ms	49.2 ms	6.5 ms
Rolling GC, 10^7 bases	134 ms	80.8 ms	44.7 ms

216× behind NumPy on the alignment. The float64 equivalents of every operation in that inner row were measured at parity or better in experiment 11.

Round one: the scan and the thread

Two `int64` arms were simply absent, and both are one-liners against paths that already existed:

- **`nd_mapthread2`** — the column fold behind `MapThread[f, {a, b}]` — was written `dtype != NDT_FLOAT64 → return NULL`. `Min` and `Max` return one of their arguments, so they are exact on an integer buffer by construction; `Plus` and `Times` go through `ci_add_i64`/`ci_mul_i64` and abandon the whole array on the first overflow, so the List path re-runs it and GMP answers exactly rather than a wrapped sum reaching the user.
- **`ndred_scan`** already had a complete exact `int64` arm. `Fold` and `FoldList` were simply not on `pack.c`'s `INT64_OK` list, so the buffer was materialised before the builtin could see it.

op, 10^6 int64	before	after	float64 control
`MapThread[Max, {a, b}]`	853 ms	13.7 ms	12.3 ms
`FoldList[Max, 0, v]`	302 ms	1.36 ms	1.01 ms

Both now match their float64 counterparts. And the alignment row moved from 11.11 s to 10.58 s.

Round two: profiling the row that did not move

A 5% improvement from two 60× and 220× primitive wins is a result in itself: it says the cost was somewhere else entirely. Timing the inner row's six expressions directly, per 2000-element call:

	before
`Abs[seq − ch]`	460 µs
`Most[prev]`	138 µs
`MapThread[Max, {Most[prev], Rest[prev]}]`	805 µs
`UnitStep[…]`	8.5 µs ✓
`FoldList[Max, seed, cand]`	2.0 µs ✓
`prev[[1]]`	0.25 µs ✓
the whole row	5.4 ms

`NDArrayQ` on the results of the first two: False. Two more heads with no `int64` arm, and they were poisoning the `MapThread` that consumed them — `MapThread` was fast, but both its operands arrived unpacked.

Abs is registered as a projection kernel (`to_real`): the result is always real, even for complex input. That is right for `Abs[z]` and wrong for `Abs[{-3, 2}]`, which is `{3, 2}` with exact `Integer` heads. The descriptor had no way to say "exact integer here, projection there", so `ndarray_map_unary` gained one line: when a `to_int` kernel has no arm for this dtype, fall through to the categories below instead of declining the whole call. `to_int` now means "prefer the exact integer answer where one exists", and `Abs` supplies an `int64` arm that abandons on `|INT64_MIN|` so the List path answers with the exact bignum.

First/**Last**/**Most**/**Rest** were given buffer paths by experiment 11 and never added to `INT64_OK`. All four are memcpys of whole rows, or a single element through `ndarray_element_to_expr`, which yields an exact `Integer` from an integer buffer. There is no arithmetic in them to be exact about.

	before	after
`Abs[v]`, 2000 int64	460 µs	7.5 µs
`Most[v]`, 2000 int64	138 µs	0.74 µs
`MapThread[Max, {Most[v], Rest[v]}]`	805 µs	5.2 µs
the whole Needleman-Wunsch row	5.4 ms	0.10 ms

Round three: the sliding window

k-mer counting barely moved either, and the profile was even more one-sided:

	cost
`Partition[code, 12, 1]`	439 ms
`… . pow4` (6×10^6 integer MACs)	5.3 ms
`Union` of 5×10^5 keys	11.7 ms

`ndstruct_partition` handled `Partition[a, k]` — the non-overlapping tiling, which is one memcpy — and declined every offset form. But the offset form is the sliding window: rolling statistics, n-grams, time-series embedding, the setup for a correlation. With $d \neq k$ the rows overlap and it is `rows` strided copies of `k` elements, which is thirty lines rather than one:

	before	after
`Partition[v, 12, 1]`, 5×10^5	439 ms	20.1 ms
the whole k-mer kernel	420 ms	25.0 ms

Only complete rows are produced, matching the List path; the padded four-argument form still declines.

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
Needleman-Wunsch, 2000 × 2000	11.11 s	186 ms	1.841 s	51.4 ms	9.91× faster than WL
k-mer encode + distinct count	492 ms	25.0 ms	49.2 ms	6.5 ms	1.97× ahead of WL
Rolling GC content, 10^7 bases	134 ms	137 ms	80.8 ms	44.7 ms	

The alignment moved 60× and crossed from 6× behind Mathematica to 9.9× ahead of it; the k-mer count moved 20× and crossed from 10× behind to 2× ahead. All three values are integers or integer sums and agree exactly across all three systems.

What is still open

- k-mer is 3.8× NumPy, and that gap is structural. NumPy's `sliding_window_view` is a stride trick: it returns a view, copies nothing, and costs O(1). Mathilda materialises 6×10^6 elements because an `NDArray` has no stride vector. This is the same missing feature as the transposed view in plan item 9.1, and closing either one properly means closing both.
- The alignment is 3.6× NumPy for the ordinary unfused-composition reason: the row is six passes over 2000 elements where a fused kernel would make two.
- **`gcwin`** at 3.1× NumPy is `Accumulate` over 10^7 int64 plus two `Drop`s; nothing in it is on a slow path, it is three passes over 80 MB.

The lesson, which is the same one three sweeps running

> A primitive fixed in isolation buys nothing if the value reaching it is already unpacked.

The scan and the thread were made 60× and 220× faster and the benchmark moved 5%, because `Abs` and `Most` upstream of them were still materialising. The fourth sweep found the same shape (`Outer`'s fast path was correct and unreachable), and the third found it twice. The only reliable way to see it is to re-profile the composition after fixing the primitive, which is what rounds two and three of this experiment are.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Needleman-Wunsch	186 ms	51.4 ms (NumPy)	3.61×	six unfused passes per row, each with per-call overhead
k-mer count	25.0 ms	6.5 ms (NumPy)	3.82×	the sliding window is materialised; NumPy's is a view
Rolling GC content	137 ms	44.7 ms (NumPy)	3.07×	three passes over 80 MB

Mathilda is ahead of Mathematica on the first two (9.9× and 2.0×) and behind NumPy on all three, by a consistent 3–4×. That consistency is the diagnosis: it is not three problems, it is one.

The road to fastest

1. Strided views. The k-mer row is the clearest case in the suite. `np.lib.stride_tricks.sliding_window_view` returns a view — it copies nothing and costs O(1) — where `Partition[code, 12, 1]` materialises 6×10^6 elements, 48 MB, to be read once by a `Dot`. A stride vector on `NDArray` that consumers honour would make the window free and hand BLAS a strided buffer directly. This is the same feature as the transposed view in experiment 17's roadmap and plan item 9.1 — one design change closes both, and it is the largest single item across the whole suite. Expected: the k-mer row from 25.0 ms to roughly 8 ms, i.e. ahead of NumPy, because the `Dot` and the `Union` are already at 4.8 ms and 10.8 ms and both have headroom.
2. Fuse the alignment row. Six whole-array passes over 2000 int64 per row — `Abs`, `UnitStep`, two shifted adds, an elementwise `Max`, a scan — where a fused kernel makes two. At 2000 elements each pass is dominated by its own call overhead rather than by memory, which is why this is worth more here than on a 10^7-element row. Expected: 186 ms toward 60 ms.
3. A **`Union`** that does not sort. 10.8 ms of the k-mer row is `Union` over 5×10^5 int64, which currently sorts and dedups. A hash-set pass is O(n) rather than O(n log n) and is the same machinery `Tally` already has. Expected: ~3 ms.
4. Thread the scan. `Accumulate` over 10^7 int64 is serial. A two-pass parallel prefix sum is standard and exact for integers (no reassociation concern, unlike float64 — which is why the int64 arm can be threaded when the float one cannot without a documented change).

Items 1 and 2 would put all three rows ahead of NumPy. Nothing here needs new mathematics; item 1 needs a representation decision.

Verification

- `test_fifth_sweep_fast_paths` covers all of it differentially — the integer `MapThread` and scan including their overflow abandonment, `Abs` on integers, reals, complex and `INT64_MIN`, the four leading-axis slices at rank 1 and 2, and `Partition` at `d < k`, `d = k`, `d > k`, one row, no rows, rank 2 and the padded 4-argument form.
- A dedicated composition case reproduces the Needleman-Wunsch substitution row and the k-mer pipeline end to end, packed against unpacked.
- All three benchmark values agree across all three systems.